Journal de bord — Stage MARTA

2026-09-22 · @Someone

Ce journal retrace, semaine après semaine, les missions réalisées, les compétences développées et les difficultés rencontrées tout au long du stage.

Mois 1 — Pipeline de données (Semaines 1 à 4)

Semaine 1. Prise en main du sujet de stage : conditionnement d'un système de trading multi-agents par des embeddings de régime de marché. J'ai passé la semaine à lire l'architecture existante (trois couches : encodeur de régime, agents LLM, exécution par renforcement) et à mettre en place mon environnement de travail (Python 3.11, conda). Ce que j'ai appris : un projet de cette taille se comprend en partant des sorties de chaque module plutôt qu'en essayant de tout lire linéairement. Ce qui m'a semblé difficile au premier abord : le volume de code et de concepts (finance quantitative, deep learning, RL) à absorber en même temps — j'ai géré cela en me concentrant d'abord sur la couche de données, la plus concrète.

Semaine 2. Mission : construire la récupération des données de marché (`fetch.py`). J'ai utilisé l'API CCXT pour récupérer l'historique horaire BTC/USDT et ETH/USDT depuis Binance, et découvert `histdata.com` pour les données forex/or au format minute. Compétence développée : lire une documentation d'API tierce et gérer la pagination d'une requête historique par lots. Difficulté rencontrée : la boucle de récupération s'arrêtait prématurément sur certains actifs à cause d'une condition d'arrêt mal calibrée (`len(batch) < 1000`) — je l'ai identifiée en comparant la dernière date récupérée à la date attendue, puis corrigée pour ne s'arrêter que sur un lot vide ou la date de fin configurée.

Semaine 3. Mission : transformer les cours bruts en indicateurs techniques exploitables (`features.py`). J'ai calculé ATR, ADX, RSI, largeur de bandes de Bollinger, MACD et OBV à l'aide de la librairie `ta` . J'ai appris à distinguer un indicateur qu'il faut normaliser (comparable entre actifs) d'un indicateur à conserver en valeur brute pour préserver un signal de choc (volatilité réalisée non normalisée, utile pour détecter un krach). Ce travail confirme mon goût pour le traitement de données quantitatives — j'aime manipuler des séries temporelles et comprendre ce que chaque transformation signifie financièrement, plutôt que de l'appliquer sans réfléchir.

Semaine 4. Mission : aligner les séries de plusieurs actifs sur un même calendrier temporel et valider la cohérence du pipeline complet (`align.py` , `validate.py`). J'ai dû gérer le fait que les marchés crypto sont ouverts 24h/24 alors que le forex ferme le week-end, ce qui limite mécaniquement le taux de recouvrement temporel entre les deux.

Organisation : j'ai écrit un rapport de validation automatique (structure, plages de valeurs, anomalies OHLCV) plutôt que de vérifier les données à l'œil, pour pouvoir relancer le contrôle à chaque mise à jour du pipeline sans repartir de zéro.

Détection de régimes de marché (Semaines 5 à 6)

Semaine 5. Mission : regrouper l'historique de marché en régimes statistiques distincts (calme, tendance, stress) à l'aide d'un modèle de Markov caché (HMM) gaussien. J'ai appris à utiliser le critère BIC pour choisir objectivement le nombre d'états du modèle plutôt que de le fixer arbitrairement, et à sous-échantillonner les données avec une covariance diagonale pendant la recherche d'hyperparamètres pour que le calcul reste praticable (le modèle final, lui, est réentraîné en covariance complète sur toutes les données). Compétence utilisée concrètement : lecture de courbes BIC pour arbitrer entre complexité du modèle et qualité d'ajustement.

Semaine 6. Mission : vérifier que les régimes détectés ont un sens financier réel (et non de simples artefacts statistiques), en calculant les statistiques de volatilité et de rendement propres à chaque régime. Réalisation concrète : un rapport de validation sémantique automatique, qui échoue si les régimes ne se distinguent pas assez en volatilité. Ce que j'en retire : un modèle qui « tourne » sans erreur ne veut pas dire qu'il produit un résultat exploitable — il faut toujours le confronter à une vérité métier indépendante du code. C'est une compétence que je n'avais jamais pratiquée avant ce stage et qui me semble transférable à n'importe quel projet de machine learning appliqué.

Système multi-agents LLM (Semaines 7 à 11)

Semaine 7. Début de la conception du système de délibération multi-agents avec LangGraph. J'ai construit une classe de base commune à tous les agents, chargée de l'appel au modèle de langage (Mistral 7B via Ollama) et surtout de la récupération de sa réponse en JSON. Difficulté : un LLM ne renvoie pas toujours un JSON strictement valide. J'ai appris à construire une chaîne de récupération en plusieurs étapes (parsing direct, extraction depuis un bloc markdown, recherche par expression régulière, réparation de virgules mal placées) plutôt que de faire échouer tout le système à la moindre réponse imparfaite — une leçon sur la robustesse à avoir face à un composant non déterministe.

Semaine 8. Mission : implémenter les trois premiers agents spécialisés — analyse macro (corrélations inter-actifs), analyse technique (momentum, indicateurs) et gestion du risque (volatilité GARCH, limites de position). Chacun devait produire une thèse structurée à partir du même contexte de marché. Compétence développée : l'ingénierie de prompt orientée sortie structurée (schémas Pydantic), pour que la réponse d'un modèle de langage reste exploitable par du code en aval.

Semaine 9. Mission : implémenter l'agent de synthèse (Fund Manager) qui agrège les avis des autres agents en une décision unique, et orchestrer un débat en deux tours (thèses puis réfutations). Première fois que j'ai vu l'ensemble du système fonctionner de bout en bout, de la donnée brute à une recommandation de trading argumentée. J'ai ressenti une vraie satisfaction à ce moment-là — c'est là que j'ai réalisé que j'aime construire des systèmes où plusieurs composants interagissent, plus que des scripts isolés.

Semaine 10. Ajout de deux agents supplémentaires : un agent de sentiment (analyse d'actualités financières via FinBERT) et un agent « contrarian » jouant systématiquement l'avocat du diable. Compétence utilisée : intégration d'un modèle de NLP pré-entraîné (FinBERT) dans un pipeline existant, avec pondération temporelle décroissante des actualités les plus récentes. J'ai appris qu'un agent « contrarian » bien conçu a plus de valeur qu'un agent supplémentaire qui serait simplement d'accord avec les autres — la diversité d'opinion étant elle-même une information utile à la décision finale.

Semaine 11. Correction de la logique de débat : le deuxième tour de réfutation ne se déclenchait pas de manière fiable quand les agents étaient réellement en désaccord. Difficulté rencontrée : déboguer un comportement conditionnel dans un système où chaque exécution produit une sortie légèrement différente (non-déterminisme du LLM) rend le diagnostic plus lent qu'avec du code classique. J'ai résolu cela en rejouant plusieurs scénarios types et en vérifiant la logique de déclenchement indépendamment du contenu généré. Le système à 5 agents est devenu stable et reproductible sur l'ensemble des cas de test.

Agent d'exécution RL et audit de fiabilité (Semaines 12 à 18)

Semaine 12. Expérimentation d'un modèle HMM indépendant par actif (plutôt qu'un modèle partagé), dans l'espoir de mieux capturer des événements extrêmes comme le krach de mars 2020. Résultat instructif mais négatif : le signal restait insuffisant pour créer un régime dédié à la crise. Apprentissage important : documenter et accepter un résultat qui infirme une hypothèse de départ fait autant avancer le projet qu'un résultat positif, à condition de comprendre précisément pourquoi ça n'a pas marché.

Semaine 13. Début de la couche d'exécution par apprentissage par renforcement : définition de l'espace d'état (embedding de régime + consensus des agents + état du portefeuille) et de la fonction de récompense. Domaine totalement nouveau pour moi — je n'avais jamais pratiqué le RL auparavant. Défi principal : comprendre en profondeur les concepts (politique, fonction de valeur, avantage) suffisamment vite pour pouvoir les implémenter correctement, sans pouvoir m'appuyer sur une expérience préalable.

Semaine 14. Implémentation de la politique PPO (tête catégorielle pour la direction, distribution Beta pour la taille de position) et des callbacks de curriculum d'entraînement progressif. Premiers entraînements lancés, encore instables (récompenses divergentes).

Organisation : j'ai appris à isoler un problème d'entraînement en observant les courbes de perte plutôt qu'en modifiant plusieurs paramètres à la fois.

Semaine 15 (bilan mi-stage). Premiers résultats RL exploitables, mais découverte que le curriculum complet (ajout progressif d'actifs) dégradait la politique sur des actifs à très faible variance intra-journalière. À ce stade du stage, je mesure une vraie évolution personnelle : au début, un résultat contre-intuitif m'aurait déstabilisé ; là, j'ai su le traiter méthodiquement pour en isoler la cause réelle (la fréquence des données) plutôt qu'un simple symptome.

Semaine 16. Conception d'une étude d'ablation à sept variantes pour isoler la contribution réelle du régime de marché et des agents LLM à la performance finale, avec un intervalle de confiance calculé par bootstrap par blocs (pour respecter l'autocorrélation des rendements financiers, contrairement à un bootstrap naïf). Compétence développée : concevoir un protocole expérimental rigoureux capable de départager une vraie contribution d'un simple bruit statistique.

Semaine 17. Audit de fiabilité de l'évaluation : découverte que les résultats n'étaient pas reproductibles d'une exécution à l'autre sur le même modèle, à cause de calculs GPU non déterministes. Difficulté vécue comme frustrante sur le moment (remettre en doute des résultats déjà obtenus), mais résolue en fixant les graines aléatoires et en forçant l'évaluation sur CPU. Cet épisode m'a beaucoup appris sur la rigueur expérimentale exigée en recherche appliquée.

Semaine 18. Investigation d'un biais plus subtil : un paramètre d'entraînement (pénalité d'inactivité) fuitait dans l'environnement de validation et gonflait artificiellement les résultats affichés. Une fois identifié, j'ai réévalué tous les checkpoints sauvegardés dans des conditions propres et corrigé le modèle retenu comme référence. Cette semaine confirme que mon goût pour la rigueur n'est pas qu'une préférence personnelle : dans ce métier, un biais non détecté peut fausser une conclusion entière.

Tableau de bord et données en temps réel (Semaines 19 à 21)

Semaine 19. Début du travail sur le tableau de bord : backend FastAPI avec une planification horaire des signaux de trading (APScheduler). Premier contact avec la partie « mise en production » du projet, très différente du travail de recherche pur mené jusque-là. Compétence développée : conception d'une API et organisation d'un backend qui doit tourner de manière fiable, en continu, sans supervision humaine constante.

Semaine 20. Mission : intégrer des données de marché en temps réel dans le tableau de bord, et distinguer une lecture historique figée d'un recalcul de régime véritablement dynamique. Difficulté : plusieurs parties du code lisaient encore une source de données «

figée » au lieu de la version « live », ce qui donnait l'illusion d'un système à jour alors qu'il ne l'était pas. J'ai appris à tracer systématiquement l'origine des données affichées à l'utilisateur final plutôt que de faire confiance à l'apparence du résultat.

Semaine 21. Série de corrections de bugs liés à la fraîcheur des données (régime de marché, flux forex/or), et mise en place d'un pont de données live pour combler un vide de plusieurs mois sur certains actifs. Organisation : j'ai priorise les corrections par impact visible pour l'utilisateur plutôt que par facilité technique, ce qui m'a demandé de redéfinir mes priorités de délivrable en cours de semaine. Ce travail confirme que le métier m'intéresse autant en recherche qu'en mise en production — je n'avais pas anticipé apprécier autant cette partie du stage avant de la pratiquer.

Finalisation du projet (Semaines 22 à 25)

Semaine 22. Génération de centaines de débats multi-agents supplémentaires (calcul externalisé sur Kaggle) pour densifier la couverture du système, puis backtests comparatifs entre la stratégie basée sur les agents et une stratégie « buy-and-hold » sur cinq crypto-actifs. Résultats mitigés mais honnêtes : gain statistiquement significatif sur deux actifs, pas sur les trois autres. Apprentissage important sur mon métier futur : savoir présenter un résultat nuancé sans l'exagérer ni le minimiser est aussi une compétence professionnelle à part entière.

Semaine 23. Développement d'un panneau « Strategy Tester » dans le tableau de bord, permettant à un utilisateur de tester différentes stratégies de trading configurables, avec courbe d'équité et journal de transactions réel. Compétence développée : concevoir une fonctionnalité pensée pour un utilisateur final non technique, et pas seulement pour moi-même en tant que développeur.

Semaine 24. Ajout de l'authentification (connexion via Google et GitHub) et refonte du graphique de prix du tableau de bord pour un rendu plus fluide et professionnel. Bon exercice de sortie de ma zone de confort recherche/machine learning vers du développement produit côté interface. J'ai appris que je peux être à l'aise sur des sujets très différents (modèles statistiques un jour, authentification web le lendemain) du moment que la méthode de travail reste la même : décomposer, tester, valider.

Semaine 25 (bilan de fin de stage). Consolidation finale des résultats du projet, mise à jour complète de la documentation technique et relecture d'ensemble du système construit depuis la semaine 1. En regardant le chemin parcouru — du simple script de récupération de données au système multi-couches complet (données, régimes, agents LLM, RL, tableau de bord en production) — je mesure une vraie progression, très loin du doute ressenti en tout début de stage. Ce stage confirme mon souhait de poursuivre vers des métiers mêlant recherche appliquée et ingénierie, sur des systèmes complexes plutôt que sur des tâches isolées.